Choosing SQL vs NoSQL The Right Database for the Job

Chapter 06 - Sub-Chapter 09

NoSQL Databases Series

The Right Mindset	Questions to ask before choosing
SQL vs NoSQL	Feature-by-feature comparison matrix
Decision Guide	Scenarios and recommended choices
Polyglot Persistence	Using multiple databases in one system

1 The Right Mindset

There is no best database — only the best fit

Junior developers ask 'Which database should I use?' as if there is one right answer. Senior engineers ask 'What does this specific system need to do, and which database matches those needs?'

The wrong database choice is expensive: migrations are painful, performance problems are hard to fix after launch, and the wrong tool forces you to fight the database instead of using it. Getting this decision right upfront saves months of pain later.

The questions to ask before choosing a database:

- **What are the most frequent read patterns?** By key? By range? By graph traversal? By text search?
- **What is the write pattern?** High frequency inserts? Occasional updates? Append-only?
- **How structured is the data?** Rigid and consistent (billing) vs. flexible and varied (product catalog)?
- **What are the consistency requirements?** Financial transactions need ACID. Sensor data tolerates eventual consistency.
- **What is the expected scale?** 10,000 users vs. 100 million users have different needs.
- **What does the team know?** A PostgreSQL team using MongoDB incorrectly is worse than using PostgreSQL correctly.
- **What is the operational model?** Self-hosted vs. managed cloud service. On-prem vs. cloud-native.

2 SQL vs NoSQL — Head to Head

Feature-by-feature comparison across database types

Database Selection Decision Matrix

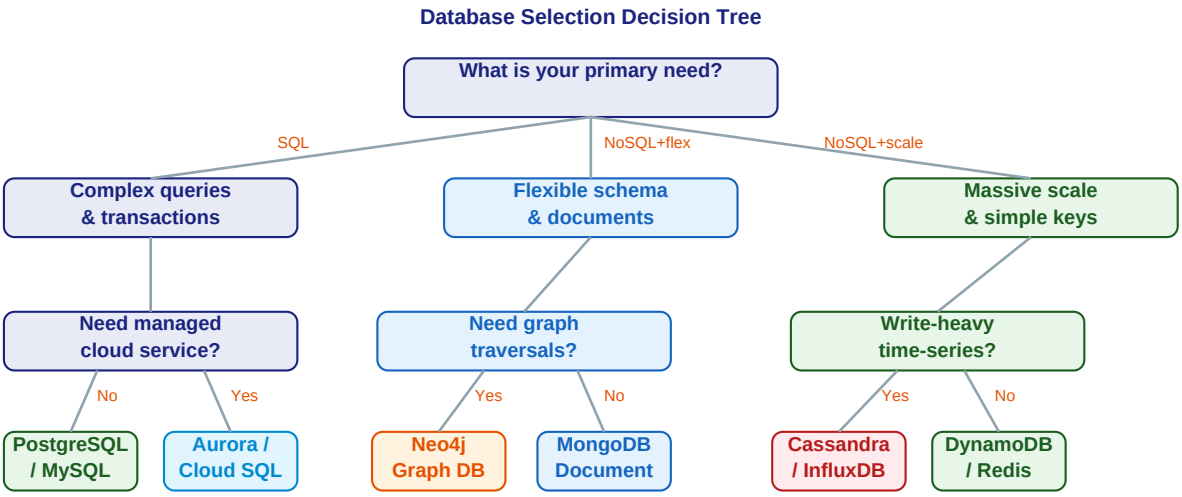
Criteria	PostgreSQL	MongoDB	Cassandra	DynamoDB	Neo4j
Schema flexibility	✗	✓	✓	✓	✓
Complex queries / JOINs	✓	~	✗	✗	✓
ACID transactions	✓	✓	✗	✓	✓
Horizontal scalability	~	✓	✓	✓	~
Write throughput	~	~	✓	✓	✗
Consistency by default	✓	~	✗	✓	✓
Operational simplicity	✓	✓	~	✓	✓
Mature tooling / ecosystem	✓	✓	✓	~	~

✓ = Strong ~ = Moderate ✗ = Weak/Not supported

Property	SQL (PostgreSQL)	Document (MongoDB)	Wide-Column (Cassandra)	Key-Value (DynamoDB)
Data model	Tables, rows, typed columns	JSON documents, nested	Rows with flexible columns	Key + value (any type)
Query power	Full SQL, JOINs, subqueries	Rich query lang, aggregation	CQL, partition key required	Key lookup, range on SK
Transactions	Full ACID, multi-table	Multi-doc ACID (4.0+)	LWT (single partition)	Multi-item ACID
Scaling	Vertical + read replicas	Horizontal sharding	Linear horizontal	Serverless auto-scale
Consistency	Strong (serializable)	Tunable per operation	Tunable ONE/QUORUM/ALL	Strong or eventual
Best for	ERP, finance, any relational	CMS, catalog, user profiles	IoT, logs, global 24/7	Session, cart, gaming

3 Decision Guide

How to choose for common scenarios



Simplified guide — consider team experience, existing stack, and operational overhead too

Common scenarios and recommended choices:

Scenario: E-commerce platform (orders, payments, inventory)

Choice: PostgreSQL

Why: Complex queries, ACID transactions, financial data, many JOINs (products, inventory, orders, users).

Scenario: Product catalog with varied attributes

Choice: MongoDB

Why: Electronics have different fields than clothing. Schema flexibility is essential. Query by category/price range.

Scenario: IoT sensor platform (10M sensors, 1M readings/sec)

Choice: Cassandra or InfluxDB

Why: Append-only writes, time-range queries by sensor ID. Cassandra for OLTP; InfluxDB for pure time-series analytics.

Scenario: User session store and rate limiter

Choice: Redis

Why: Sub-millisecond reads, TTL expiry, simple key-value access pattern. Not a primary database.

Scenario: Social network (friends, recommendations, connections)

Choice: Neo4j + PostgreSQL

Why: Neo4j for graph traversals (friends-of-friends, recommendations). Postgres for user account data and billing.

Scenario: Global gaming leaderboard and player state

Choice: DynamoDB

Why: Known access patterns (by player ID), serverless scaling for launch spikes, multi-region active-active.

Scenario: Application metrics and dashboards

Choice: Prometheus + Grafana

Why: Pull-based metrics scraping from Spring Boot Actuator. InfluxDB if push-based or longer retention needed.

4 Polyglot Persistence

Use multiple databases — each for its strength

Modern microservices architectures often use multiple databases within the same application — each service picks the best database for its specific needs. This is called polyglot persistence.

Real-world example: an e-commerce platform using 5 databases:

```
// Microservices architecture -- each service owns its database

// 1. Order Service         --> PostgreSQL
//    Reason: ACID transactions, financial data, complex queries
//    Data: orders, payments, refunds, tax records

// 2. Product Catalog       --> MongoDB
//    Reason: flexible schema (each product type has different attributes)
//    Data: product descriptions, images, specs, variants

// 3. Session / Auth        --> Redis
//    Reason: sub-ms reads, TTL expiry, simple key-value
//    Data: JWT tokens, user sessions, login rate limits

// 4. Search                --> Elasticsearch
//    Reason: full-text search, faceted filtering, relevance scoring
//    Data: product names, descriptions, tags (indexed from MongoDB)

// 5. Analytics / Metrics   --> ClickHouse + Prometheus
//    Reason: OLAP queries on billions of events, application metrics
//    Data: clickstream, funnel analysis, API latency, error rates

// Each team chooses independently -- no single database for everything
```

Polyglot persistence trade-offs:

Benefits:

✓ Each service uses the optimal database for its access patterns — better performance.

✓ Teams are independent — the Order team can upgrade PostgreSQL without affecting the Catalog team.

✓ Failure isolation — a Redis outage affects sessions but not orders.

Trade-offs:

✗ Operational complexity — your team needs to manage and monitor multiple database technologies.

✗ Cross-service queries are harder — no JOINs across databases. Use event streaming (Kafka) to sync.

✗ Consistency across services — distributed transactions are complex. Use Saga pattern instead.

Key Rules

- Start with PostgreSQL for any new project -- it handles 90% of use cases correctly.
- Add a specialized database only when PostgreSQL proves inadequate -- not preemptively.
- The team's familiarity with a database matters as much as the database's raw capabilities.
- Managed services (Atlas, DynamoDB, Aurora) reduce operational cost -- prefer them over self-hosted.
- Polyglot persistence is powerful but adds complexity -- earn it through demonstrated need.